

Operating Systems

Youjip Won

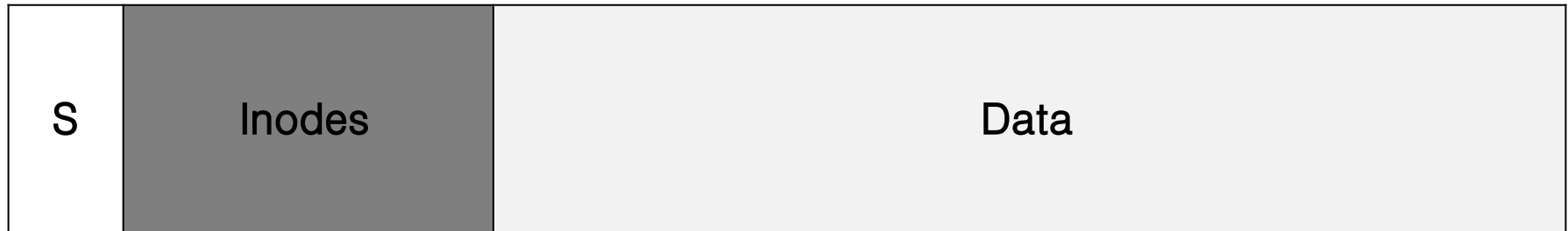
KAIST



41. Locality and The Fast File System

Overview

- The First UNIX File system:



Data structures

- Simple and supports the basic abstractions.
- Easy to use file system.

However, Poor Performance !

- In this chapter, we study how the first unix filesystem can become fast.

Problem of Unix operating system

- Unix file system treated the disk as a **random-access memory**.
- Example of random-access blocks with Four files.
 - ◆ Data blocks for each file can accessed by going back and forth the disk, because they are are **contiguous**.

A1	A2	B1	B2	C1	C2	D1	D2
----	----	----	----	----	----	----	----

- ◆ File b and d is deleted.

A1	A2			C1	C2		
----	----	--	--	----	----	--	--

- ◆ File E is created with free blocks. (**spread across** the block)

A1	A2	E1	E2	C1	C2	E3	E4
----	----	----	----	----	----	----	----

FFS: Disk Awareness is the solution

- ▣ FFS is **Fast File system** designed by a group at Berkeley.
- ▣ The design of FFS is that file system structures and allocation policies to be “disk aware” and improve performance.
 - ◆ Keep same API with file system. (`open()`, `read()`, `write()`, etc)
 - ◆ Changing the internal implementation.

Cylinder Group

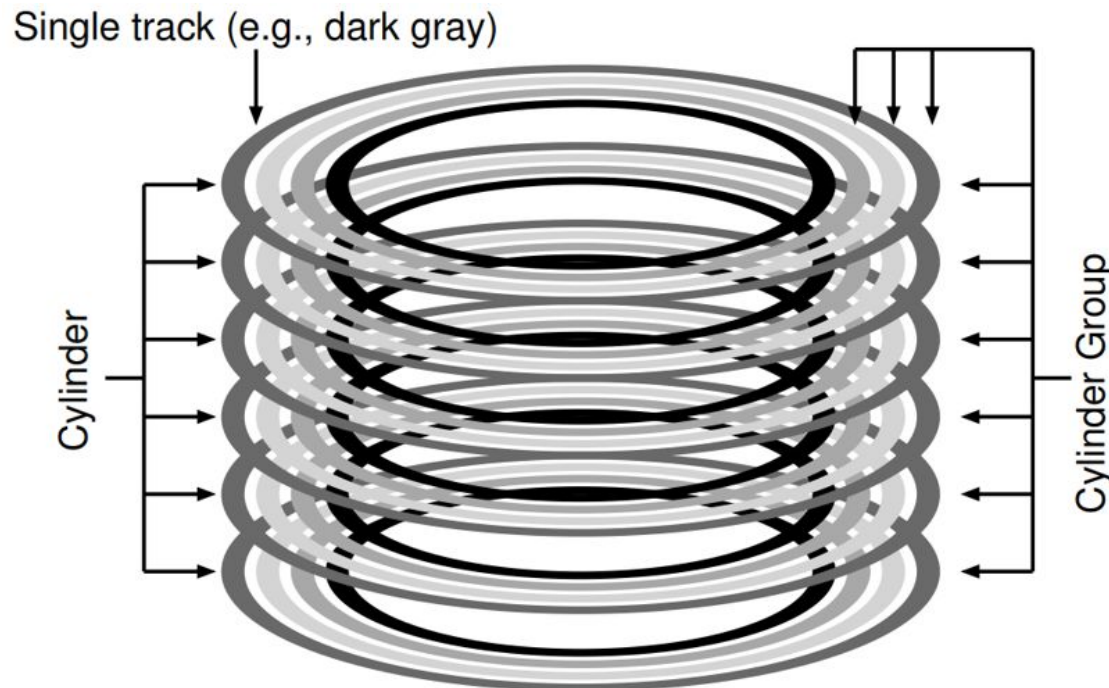
- FFS divides the disk into a bunch of groups. (**Cylinder Group**)
 - ◆ Modern file system call cylinder group as block group.
- These groups are used to improve seek performance.
 - ◆ By placing two files within the same group.
 - ◆ Accessing one after the other **will not be long seeks** across the disk.
 - ◆ FFS needs to allocate the files and directories within each of these groups.



- Data structure for each cylinder group.
 - ◆ A copy of the super block for reliability reason.
 - ◆ inode bitmap and data bitmap to track free inode and data block.
 - ◆ inodes and data block

Cylinder Group (Cont.)

- ▣ **Cylinder** : Tracks at same distance from center of drive across different surfaces.
 - ◆ All tracks with same color
- ▣ **Cylinder Group** : Set of N consecutive cylinders
 - ◆ if $N=3$, first group does not include black track



How To Allocate Files and Directories?

- Policy is “**keep related stuff together**”
- The placement of directories
 - ◆ Find the cylinder group with a low number of allocated directories and a high number of free inodes.
 - ◆ Put the directory data and inode in that group.
- The placement of files.
 - ◆ Allocate data blocks of a file in the same group as its inode
 - ◆ It places all files in the same group as their directory

How To Allocate Files and Directories? (Example)

- ▣ In the example
 - ◆ there are 10 groups, and each group have 10 inodes and 10 data blocks.
 - ◆ there are three directories: '/', '/a', and '/b'
 - ◆ there are four files: '/a/c', '/a/d', '/a/e', and '/b/f'

Filesystem is described as

below:

<Table representing Filesystem>		<Description of Table>	
group	inodes	data blocks	
0	/-----	/-----	In group 0, an inode and a data block for '/' are allocated
1	acde-----	acde-----	There are nine free inodes and data blocks In group 1, inodes for file 'a', 'c', 'd', and 'e' is allocated, one data block for file 'a', 'c', 'd', and 'e' are allocated.
2	bf-----	bff-----	
3	-----	-----	
4	-----	-----	
5	-----	-----	In group 2, inodes for file 'b' and 'f' are allocated, one There are six free inodes and three free data blocks data block for file 'b', two data blocks for file 'f' are allocated.
6	-----	-----	
7	-----	-----	
8	-----	-----	
			There are eight free inodes and seven free data blocks

How To Allocate Files and Directories? (Example)

group	inodes	data blocks
0	/-----	/-----
1	acde-----	acdde-----
2	bf-----	bff-----
3	-----	-----
4	-----	-----
5	-----	-----
6	-----	-----
7	-----	-----
8	-----	-----

< With FFS policies

□ With FFS policies

- ◆ the data blocks of each file are near each file's inode
- ◆ files in the same directory are near one another.

□ Without FFS policies (Simply spread directories and files across groups)

- ◆ Access to files '/a/c', '/a/d', and '/a/e' now spans three groups instead of one as per the FFS approach.

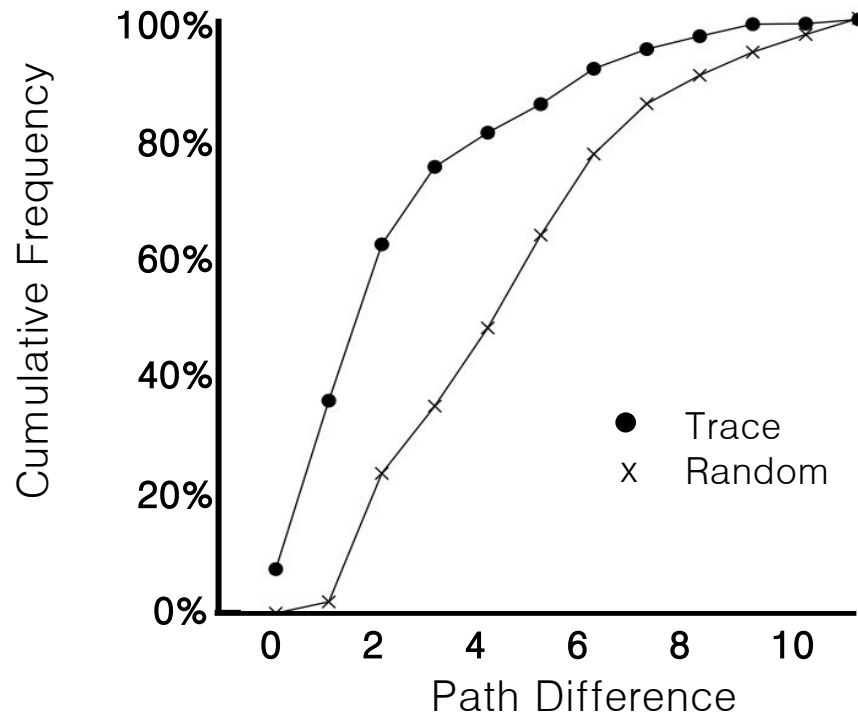
group	inodes	data blocks
0	/-----	/-----
1	a-----	a-----
2	b-----	b-----
3	c-----	cc-----
4	d-----	dd-----
5	e-----	ee-----
6	f-----	ff-----
7	-----	-----
8	-----	-----

< Without FFS policies

>

FFS Locality for SEER Traces.

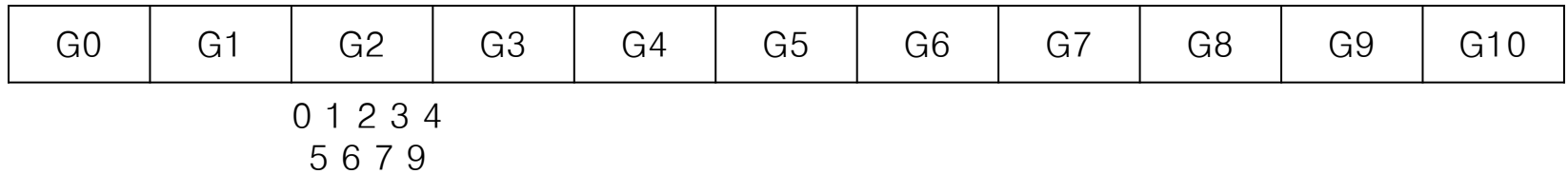
- SEER traces is analysis about how “far away” file accesses were from one another in the directory tree.
- Path difference: how far up the directory tree you have to travel to find the common ancestor of two files



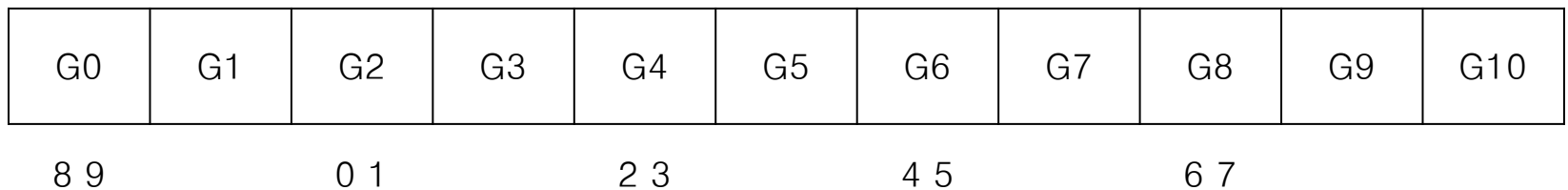
The Large-File Exception

- FFS without the large file exception handling.

G: block group

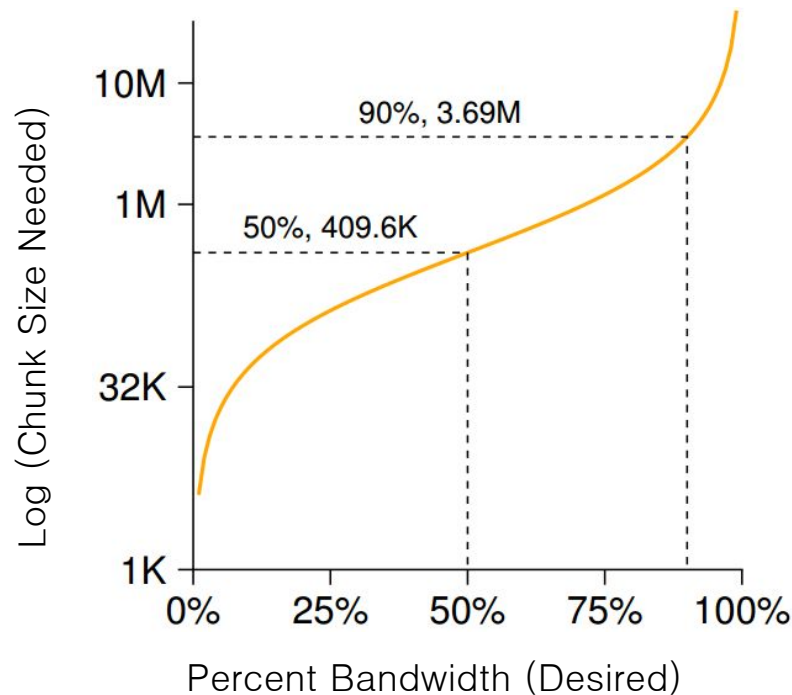


- FFS with the large file exception handling.
 - ◆ Spread blocks of a file across the disk.
 - ◆ Consider the time transferring big data from disk and block seek time.



The Large-File Exception (Cont.)

- How big do chunks have to be?
 - ◆ First twelve direct blocks places in same group as the inode
 - ◆ Indirect block, and all blocks it point places in a different group.



A few other things about FFS

- ▣ Internal fragmentation
- ▣ Sub-blocks
 - ◆ Ex) Create a file with 1 KB : use two sub-blocks, not an entire 4-KB blocks
- ▣ Parameterization
- ▣ Track buffer
- ▣ Long file names
 - ◆ Enabling more expressive names in the file system
- ▣ Symbolic link

Summary

- ▣ The introduction of fast file system (FFS)
 - ◆ It finds that the problem of file management is one of the most. important issue within an operating system.
 - ◆ It makes file system fast, considering characteristic of disk.
- ▣ Many file systems take cues from FFS.
 - ◆ ex) ext4, ext3, ext4